Thomas Nguyen
11888002
Professor Zhe Dang
Due 2/23/26

---

1. (easy) Write an algorithm that selects both the maximal element and the minimal element from an array A of n elements, using only 1.5·n comparisons.

Scan through array *A* from left to right and maintain two variables such called *max* and *min*. While, reading store *a* and *b* from the array at a time and compare them in order to refresh the new *max* and new *min*

- Compare *a* and *b*
- Compare *a* with *max* , set larger one to *max*
- Compare *b* with *min*, set smaller one to *min*

3 Comparisons are used per pair
There are n/2 pairs

so finding both min and max takes

$$\frac{3n}{2} - 2 \text{ comparisons}$$

2. (not so easy) The algorithm S(A, n, i) selects all the j-th smallest elements (with j ≤ i) from an array A of n elements, by using linearselect to select each of the j-th smallest elements (with j ≤ i). Clearly, one could also implement S alternatively as T (A, n, i), which first sort A (on average-case and on worst- case, the sorting takes time O(n log n) using mergesort) and then select the first i elements. Please compare the average-case complexities of the two algorithms; i.e., For the average-case complexities, under what onditions (on the choices for i), S is better than T or vice versa.

Alg S : LinearSelect avg time is O(i * n)
A;g T : Merge sort (O n log n) + O(i) = O(n log n)

When O(i * n) >= O(n log n) its when i >= O(log n)

3. (hard) In class, we have demonstrated the worst case complexity analysis for linearselect where each group has k = 5 numbers. Please show the worst case complexities for k = 3 and k = 7.

Case k = 3, total number of medians is $\frac{n}{3}$. At least $2 * \frac{1}{2} * \frac{n}{3} = \frac{n}{3}$ elements is less than the median of the median. So, the worst case location of the median of the $n - \frac{n}{3} = \frac{2n}{3}$ so,

$$T_W(n) = O(n) + T_W(\frac{n}{3}) + T_W(\frac{2n}{3})$$

Case k = 7, total number of medians is $\frac{n}{7}$. At least $4 * \frac{1}{2} * \frac{n}{7} = \frac{2n}{7}$ elements is less than the median of the median. So, the worst case location of the median of the $n - \frac{2n}{7} = \frac{5n}{7}$ so,

$$T_W(n) = O(n) + T_W(\frac{n}{7}) + T_W(\frac{5n}{7})$$

4. (hard) Let ilselect(A, n, i) be an algorithm that selects the i-smallest from an array A with n integers. It works as follows:

```
ilselect(A, n, i){
r=partition(A, 1, n);
//test if A[r] is the element to be selected
if i == r, return A[r];
//test if quickselect from the low-part
if i < r, return quickselect(A, 1, r - 1, i);
//test if linearselect from the high-part
if i > r, return linearselect(A, r + 1, n, i - r);
}
```

That is, the algorithm runs quickselect on the low-part or runs linear select on the high-part. Show the worst-case complexity and the average complexity of the algorithm.

Partition

LOW	|	HIGH
r-1	r	n-r

- if i == r, ret A(r)
- if i < r , quickSelect for LOW
- if i > r , linearSelect from HIGH

Let $T_{AVG}(n)$ be average time complexity of ilselect
// A(1...n) is rand, i,r are random
// Fix r. Let i be random. Then r be random
// Prob (i == r) = 1/n
// Prob (i < r) = (r-1)/n
// Prob (i > r) == (n-r)/n

$$T_{AVG}(n) = \frac{1}{n}\sum_{r=1}^{n}\{\frac{1}{n}O(1) + \frac{r-1}{n}O(r-1) + \frac{n-r}{n}O(n-r)\} + O(n)$$

$$= \frac{1}{n}\sum_{r=1}^{n}\{\frac{1}{n}a + \frac{r-1}{n}a(r-1) + \frac{n-r}{n}a(n-r)\} + an$$

$$= \frac{a}{n}\sum_{r=1}^{n}\{\frac{1}{n} + \frac{(r-1)^2}{n} + \frac{(n-r)^2}{n}\} + an$$

$$= \frac{a}{n^2}n + \frac{a}{n^2}\sum_{r=1}^{n}(r-1)^2 + + \frac{a}{n^2}\sum_{r=1}^{n}(n-r)^2 + an$$

$$= \frac{2a}{n^2}\sum_{r=1}^{n}(r-1)^2 + an$$

$$<= \frac{2a}{n^2}\int_0^n x^2 dx \ + an$$

$$= \frac{2a}{n^2} * \frac{1}{3}n^3 + an = O(n)$$

Let $T_W(n)$ be worst time complexity of ilselect

$$T_W(n) = max_{1<=r<=n}\{O(r-1)^2), O(n-r)\} + O(n)$$

$$<= max\{max_{1<=r<=n}a(r-1)^2, max_{1<=r<=n}\} + an$$

$$= max\{a(n-1)^2, a(n-1)\} + an$$

$$a(n-1)^2 + an = O(n^2)$$

5. We use 5com to denote an operation that sorts 5 numbers. Show the minimal number of 5com operations that one need to sort n distinct numbers.

5com is a device to sort 5 numbers. We only use this device to sort n given numbers in an array. Before the array is sorted, the amount of unknown info on the ordering (of the n numbers) is $\log_2 n!$ bits. Each use of 5com will reduce the unknown info by *at most* $\log_2 5!$ bits. AT the end of sorting, the ordering is known and hence the unknown info becomes 0. So, the minimal # of uses of 5com device is *at least*

$$\frac{\log_2 n!}{\log_2 5!} = \log_{120} n!$$